Exercise Sheet 2: Arrays, plotting and benchmarking

∘ Self-study † Class discussion

Class discussion: Questions 1, 4, 7, 8.

Use the supplied odense-weather.csv for the weather question. Keep it beside the notebook. Use Python 3 with NumPy, Matplotlib and pandas. All code needed for the final question is supplied in benchmark.py.

Question 1 - A discontinuous function †

Plot $f(x) = 1/(x-1)$ on $[-2, 6]$.

(a) First plot it using one evenly spaced grid that does not contain 1. What happens near $x = 1$? Does the connecting line represent the function correctly?

(b) Split the grid into $[-2, 1)$ and $(1, 6]$ and plot the two branches separately. You can use `np.linspace(-2, 0.99, 300)` and `np.linspace(1.01, 6, 300)`. Do not evaluate the function at 1.

(c) Use a dashed magenta line of width 4. Set the x-limits to $[-2, 6]$ and the y-limits to $[-6, 6]$. Label both axes and give the plot a title. Save it as `reciprocal.png`.

Question 2 - Subplots ∘

Plot $\sin(x)$, $\sin(2x)$, $2\sin(x)$ and $2\sin(2x)$ on $[0, 2\pi]$, in a 2×2 grid of subplots.

(a) Use the same x- and y-limits for all panels: $[0, 2\pi]$ and $[-2, 2]$. Give each panel its function as a title and give the whole figure a title.

(b) Use green solid, red dashed, blue dashed and magenta dotted lines, respectively. Use `fig, axs = plt.subplots(2, 2)`; `axs[0, 1]` selects the top-right panel.

(c) Which change affects amplitude, and which affects the number of oscillations? Use the plots to explain.

Question 3 - Matrix powers ◦

Define the following matrix as a NumPy array:

```
A = np.array([[3, 1, 4],
              [1, 5, 9],
              [-5, 3, 1]])
```

Write a function that returns $-A^3 + 9A^2 - 15A$. Here $A^2 = AA$ and $A^3 = AAA$ are matrix products, not entrywise powers. Use `np.dot` or `@` for matrix multiplication. What matrix do you obtain?

This is an example of the Cayley-Hamilton theorem: a square matrix satisfies its own characteristic polynomial. No proof of the theorem is required.

Question 4 - Constructing matrices †

Construct two 7×7 arrays without writing out all 49 entries of either array.

(a) Let $A_{ij} = 1$ when $j \geq i$, and $A_{ij} = 0$ otherwise, using indices $i, j = 0, \ldots, 6$. Thus A has ones on and above the diagonal and zeros below it.

(b) Let $B_{ij} = 5$ when $j > i$, and $B_{ij} = -1$ otherwise. Thus B has fives strictly above the diagonal and minus ones on and below it.

(c) Calculate the matrix product ABA, convert the resulting array to `np.int64`, and return it from a function. Check the shapes before multiplying.

You may find `np.ones`, `np.triu` and `np.tril` useful. Look up how their diagonal-offset argument works; this is important for distinguishing "above" from "on and above" the diagonal.

Question 5 - Array toolkit ◦

Use NumPy for the following tasks. Display the results and shapes so you can check your work.

(a) Create 3×4 arrays of zeros and ones, a 4×4 identity matrix, and a 4×4 diagonal matrix with diagonal entries 1, 2, 3, 4. Inspect their data types. Also construct the integers 0 through 4 and the values 0, 0.5, 1, ..., 5.

(b) Create a 3×4 array of uniform random values in $[0, 1)$ and another of random integers from 1 through 10 inclusive. Use a NumPy random generator. Be explicit about the endpoints.

(c) Construct `C = np.arange(1, 10).reshape(3, 3)`. Select entry `(0, 0)`, all rows from the second onwards, the second row, and the third column. What shape does each selection have? Inspect `C.dtype`, then create a floating-point version with `astype(np.float64)`.

(d) Flatten `C` using both `reshape(-1)` and `ravel()`. For this contiguous array these results share its data. Change one entry of a flattened result and inspect `C`. Repeat starting from a fresh `C`, but make an explicit `.copy()` before changing the flattened result. Explain the difference.

(e) For two arrays `X` and `Y` of shape `(3, 4)`, concatenate them vertically and horizontally and predict the shapes first. Add a column to the identity matrix using horizontal stacking. Insert a new row between the first and second rows of `C`; `np.insert(..., axis=0)` may help.

(f) Display `C.T`. Using a fresh `C`, calculate `C` times its transpose with matrix multiplication. Contrast this with `C * C`, which multiplies corresponding entries. Use `np.linalg.matrix_rank` to find the rank of `C`.

(g) Create these two matrices. Use rank to decide which has an inverse, then calculate the inverse only where it exists. Verify the result by multiplying with the original matrix and comparing with the identity using `np.allclose`. Explain why the other matrix has no inverse.

```
P = np.array([[-1.0, 1.5], [1.0, -1.0]])
Q = np.array([[-1.0, 1.5], [2.0 / 3, -1.0]])
```

Question 6 - Seeds and reproducibility ◦

(a) Create a generator and draw 20 zeros and ones:

```
rng = np.random.default_rng(587)
first = rng.integers(0, 2, size=20)
```

The upper bound 2 is excluded. Make a second draw with the same generator and save it as `second`. Are the sequences identical?

(b) Create a new generator with seed 587 and make one draw of length 20. Compare it with `first` using `np.array_equal`. Explain the difference between drawing again from an existing generator and creating a new generator with the same seed.

(c) Repeat with a different seed, then with `np.random.default_rng()` without a supplied seed. Which setup would you use to make a calculation reproducible? A different seed does not guarantee different outcomes in every possible experiment, but normally changes the sequence in this example.

(d) Suppose a loop creates a new generator with seed 587 at every iteration and draws one value each time. Predict what happens, then test it. How would you change the code to generate a reproducible sequence without restarting the generator at every iteration?

Question 7 - Rain and temperature range †

Use `odense-weather.csv` to investigate a different question: in July 2024, did days with more rain tend to have a smaller daily temperature range? The range is maximum minus minimum temperature. Use this starting point; you do not need to repeat the lecture's data-inspection checklist.

```
weather = pd.read_csv("odense-weather.csv", parse_dates=["date"])
weather = weather.set_index("date").sort_index()
```

(a) Add a `temp_range` column and select July 2024. Define a wet day as one with at least 1 mm of rain. Produce a small table comparing the number of days and mean temperature range for wet days and days below this threshold. Which group has the smaller mean range?

(b) Make a scatter plot with daily rainfall on the x-axis and temperature range on the y-axis for July 2024. Mark the 1 mm threshold with a vertical line. Does the plot support the group comparison, and is there substantial variation within the groups? Label axes with units. State an observation without claiming that rain alone causes the difference.

(c) Now use all of 2024. Calculate the fraction of wet days in each calendar month and show the twelve fractions in a bar chart. Divide each month's wet-day count by the number of days recorded in that month; do not divide by the year's total. Which month has the largest fraction? Why is this a different question from comparing total rainfall or raw counts of wet days?

Data source: Copernicus ERA5 reanalysis via Open-Meteo, CC BY 4.0. These are gridded estimates near Odense. The accompanying `DATA-SOURCE.md` gives the source URL and extraction details.

Question 8 - Benchmarking matrix multiplication †

Important preparation for Assignment 1. Benchmarking measures the running time of a specified calculation. Here both methods compute the same matrix product: one uses Python loops and the other uses NumPy's `np.dot`. The goal is to check correctness, compare speeds and see how times change with matrix size.

Use the supplied `benchmark.py`. It includes the loop multiplication function and all timing code. You do not need to implement classes, matrix multiplication, timers or decorators for this question.

(a) Open the supplied file. Locate `matmul_loop`, `mult_loop` and `mult_numpy`. For inputs of shape (m, k) and (k, n), the product has shape (m, n). Its entry (i, j) is the dot product of row i and column j. Use these small arrays to check the supplied loop function against `np.dot` with `np.allclose`. Predict the shape and calculate entry `(0, 1)` by hand first.

```
A = np.array([[1, 2, 3], [4, 5, 6]], dtype=float)
B = np.array([[1, 0], [0, 1], [1, 1]], dtype=float)
```

(b) Run `python3 benchmark.py` from your Sheet 2 folder. It creates matching random square matrices, checks that results agree, times each method and saves `benchmark-results.csv`. The settings at the top control matrix sizes, batches, calls per batch and seed. Add or change sizes to investigate the trend. Reduce sizes if runs become slow. Leave the multiplication and timing implementations unchanged.

(c) Understand the timing output. A `Timer` measures one batch of calls; `our_timer` repeats batches and selects the fastest. The script defaults to three batches of three calls. Printed times are batch totals; the CSV contains the fastest total divided by calls per batch. It therefore records a per-call average. Input creation, result checks, printing and CSV writing are outside the timed calculation. Why do these distinctions matter?

(d) Write your own plotting cell or script to read the CSV and plot `loop_seconds` and `numpy_seconds` against `n`, with a legend. Plot on ordinary axes and then on log-log axes, using `ax.set_xscale("log")` and `ax.set_yscale("log")`. The horizontal variable is matrix width n, not the n^2 entries in each matrix. Interpret `speed_ratio`: loop time divided by NumPy time.

(e) Compare with another timing method, `timeit`, on one fixed size. In a separate script or notebook, import the supplied raw function with `from benchmark import matmul_loop`, then use the pattern below. Create `A` and `B` before timing. Repeat for NumPy by changing the callable to `lambda: np.dot(A, B)`.

```
import timeit
times = timeit.repeat(lambda: matmul_loop(A, B),
                      number=3, repeat=3)
print(min(times) / 3)
```

`lambda` supplies a no-argument callable for the timer to run later. In IPython/Jupyter, `%timeit -n 3 -r 3 matmul_loop(A, B)` is another option; it does not work in an ordinary Python script. Its usual summary reports a mean and standard deviation across repeats, so do not assume it reports the fastest batch like our script. Do the methods give the same broad speed comparison? Exact times need not match.

(f) For a power law $t(n) = cn^p$, log-log axes give $\log t = \log c + p \log n$. Do your curves look approximately straight? Discuss slopes, timing variation and speed ratios without treating a short experiment as proof of an asymptotic complexity bound.

Change only the supplied script's settings. Write your own plotting and analysis code in a separate cell or script.